

Práctica 03: construcción automática

Ingeniería de Software II | Repositorio: GinoSebastianD/IS2-PC3-GG | 12/09/2026

1. Objetivo y configuración de la herramienta

Construir un proyecto Java mediante un modelo de Maven que centralice la identificación del software, las opciones de compilación y las dependencias. El proyecto implementa una calculadora de promedio simple de notas de 0 a 20 y permite comprobar la compilación, las pruebas y el empaquetado.

Se seleccionó **Apache Maven 3.9.11** por su configuración declarativa en XML y su estructura convencional. El entorno comprobado es Windows 11 con **JDK 21.0.12.1**. Se fija Java 17 como versión de destino del compilador. El Wrapper conserva la versión de Maven utilizada por el proyecto [1, 2].

Configuración paso a paso. Instalar un JDK 17 o posterior desde su proveedor oficial; establecer JAVA_HOME en la carpeta del JDK y añadir bin a PATH. Abrir PowerShell, comprobar Java, clonar el repositorio y consultar la versión de Maven:

```
java -version
javac -version
git clone https://github.com/GinoSebastianD/IS2-PC3-GG.git
cd IS2-PC3-GG
.\mvnw.cmd -v
```

IS2-PC3-GG / VISOR DE EVIDENCIAS / 01

Herramientas del entorno local

Salida registrada al ejecutar .\mvnw.cmd -v en Windows 11.

01-entorno.log

```
Apache Maven 3.9.11 (3e54c93a704957b63ee3494413a2b544fd3d825b)
Maven home: C:\Users\ginod\OneDrive\Documents\ChatGPT\Molecular\IS2-PC3-GG\.m2\wrapper\dists\apache-maven-
3.9.11\03d7e36a140982eea48e22c1dcac01d8862b2550b2939e09a0809bbc5182a5bc
Java version: 21.0.12.1, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-21.0.12.1
Default locale: es_PE, platform encoding: UTF-8
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
```

Captura del visor del registro real de ejecución. Maven 3.9.11, JDK 21, compilación destinada a Java 17.

Figura 1. Captura del visor de la salida real de Maven: versión, JDK y sistema operativo.

Incidencia resuelta. La descarga inicial del Wrapper falló por una conexión TLS del entorno. Se descargó Maven desde Maven Central, se verificó su SHA-512 y se preparó la caché local. Después se ejecutó el Wrapper original. El registro del fallo y la explicación están en el repositorio.

2. Modelo de construcción del proyecto

El archivo **pom.xml** identifica el artefacto como **pe.edu.is2:calculadora-notas:1.0.0** y define el empaquetado JAR. La propiedad `maven.compiler.release=17` solicita clases compatibles con Java 17; UTF-8 fija la codificación. JUnit Jupiter 5.11.4 tiene alcance `test` y se utiliza únicamente durante las pruebas.

IS2-PC3-GG / VISOR DE EVIDENCIAS / 02

Modelo de construcción: pom.xml

Identidad del proyecto, opciones del compilador y dependencia de pruebas.

pom.xml · extractos de las líneas 6 a 10 y 14 a 28

```
06 <groupId>pe.edu.is2</groupId>
07 <artifactId>calculadora-notas</artifactId>
08 <version>1.0.0</version>
09 <packaging>jar</packaging>
10 <name>Calculadora de notas - Practica 03</name>
14 <properties>
15   <maven.compiler.release>17</maven.compiler.release>
16   <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
17   <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
18   <junit.version>5.11.4</junit.version>
19 </properties>
20
21 <dependencies>
22   <dependency>
23     <groupId>org.junit.jupiter</groupId>
24     <artifactId>junit-jupiter</artifactId>
25     <version>${junit.version}</version>
26     <scope>test</scope>
27   </dependency>
28 </dependencies>
```

Captura del contenido real de pom.xml. El archivo completo también configura los plugins y la clase principal del JAR.

Figura 2. Extractos del pom.xml real con información del proyecto, compilador y dependencia de pruebas.

La estructura separa producción y pruebas. Maven localiza ambos conjuntos de archivos sin declarar rutas manuales [1].

```
pom.xml
mvnw / mvnw.cmd
.mvn/wrapper/maven-wrapper.properties
src/main/java/pe/edu/is2/notas/
  App.java
  CalculadoraNotas.java
src/test/java/pe/edu/is2/notas/
  CalculadoraNotasTest.java
```

Plugins configurados: Clean 3.4.0, Resources 3.3.1, Compiler 3.13.0, Surefire 3.5.2, JAR 3.4.2 e Install 3.1.3. El plugin JAR agrega la clase principal `pe.edu.is2.notas.App` al manifiesto; por ello se puede utilizar `java -jar`. No se necesitan dependencias externas en tiempo de ejecución.

3. Construcción automática y resultados

Desde la raíz del proyecto se ejecutó el siguiente comando. La opción -B desactiva la interacción y -ntp reduce los mensajes de progreso de descarga:

```
.\mvnw.cmd -B -ntp clean install
```

Secuencia: clean elimina los productos previos; el ciclo solicitado por install valida el modelo, compila, ejecuta las pruebas, empaqueta, recorre verify e instala el artefacto en el repositorio local [3]. No fue necesario compilar cada archivo Java manualmente.

IS2-PC3-GG / VISOR DE EVIDENCIAS / 03

Construcción automática completada

Comando ejecutado: .\mvnw.cmd -B -ntp clean install

Compilación · extracto de 02-construccion.log

```
[INFO] --- compiler:3.13.0:compile (default-compile) @ calculadora-notas ---
[INFO] Compiling 2 source files with javac [debug release 17] to target\classes
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ calculadora-notas ---
[INFO] Compiling 1 source file with javac [debug release 17] to target\test-classes
```

Pruebas, empaquetado e instalación · tramo final del registro

```
[INFO] Results:
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jar:3.4.2:jar (default-jar) @ calculadora-notas ---
[INFO] Building jar: C:\Users\ginod\OneDrive\Documentos\ChatGPT\Molecular\IS2-PC3-GG\target\calculadora-notas-1.0.0.jar
[INFO] --- install:3.1.3:install (default-install) @ calculadora-notas ---
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 29.446 s
[INFO] Finished at: 2026-09-12T16:53:14-05:00
[INFO] -----
```

Captura de extractos del registro real. La salida íntegra se conserva en 02-construccion.log; no se modifican sus resultados.

Figura 3. Captura de extractos de la construcción real: compilación para Java 17, pruebas y BUILD SUCCESS.

Resultado comprobado: se compilaron dos archivos de producción y uno de pruebas. Surefire ejecutó **11 pruebas, sin fallos, errores ni omisiones**. Maven creó el JAR y finalizó correctamente en **29.446 segundos**, el 12/09/2026 a las 16:53:14 (UTC-5).

El artefacto es **target/calculadora-notas-1.0.0.jar**; los reportes quedan en **target/surefire-reports/**. En esta ejecución se aisló el repositorio Maven en **.m2/repository** mediante MAVEN_OPTS. En una configuración normal se utiliza el repositorio del usuario, habitualmente **~/m2/repository**.

El comando install registra el artefacto localmente. La publicación del código en GitHub es una operación independiente realizada con Git.

4. Pruebas automatizadas y ejecución del JAR

La suite verifica el promedio de varias notas, decimales, límites 0 y 20, una sola nota, lista vacía, lista nula y cinco valores inválidos: -1, 20.1, NaN e infinitos positivo y negativo. Se utilizan aserciones numéricas y comprobación de excepciones; los cinco casos inválidos corresponden a una prueba parametrizada [4].

IS2-PC3-GG / VISOR DE EVIDENCIAS / 04

Pruebas y ejecución del archivo JAR

Verificación de resultados válidos y rechazo de una nota fuera de la escala.

Reporte de Surefire · 04-pruebas.log

```
-----
Test set: pe.edu.is2.notas.CalculadoraNotasTest
-----
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.173 s -- in pe.edu.is2.notas.CalculadoraNotasTest
```

Comprobaciones de consola · 03-ejecucion.log

```
Comprobaciones del JAR - 2026-09-12 16:54:43 -05:00
PS> java -jar target/calculadora-notas-1.0.0.jar 14 16 18
CALCULADORA DE NOTAS | IS2 - PRACTICA 03
Notas procesadas: 3
Promedio: 16.00
Codigo de salida: 0
PS> java -jar target/calculadora-notas-1.0.0.jar 12.5 15
CALCULADORA DE NOTAS | IS2 - PRACTICA 03
Notas procesadas: 2
Promedio: 13.75
Codigo de salida: 0
PS> java -jar target/calculadora-notas-1.0.0.jar 21
Error: Cada nota debe ser un numero entre 0 y 20.
Codigo de salida: 1
```

Captura de registros reales guardados durante la ejecución. Los códigos de salida se registraron después de cada proceso Java.

Figura 4. Reporte de Surefire y capturas del registro de ejecución del JAR con entradas válidas e inválidas.

Comprobación funcional: las notas 14, 16 y 18 producen 16.00; las notas 12.5 y 15 producen 13.75. Ambas ejecuciones terminan con código 0. La nota 21 se rechaza y el proceso termina con código 1. Estas verificaciones confirman que el archivo empaquetado se puede ejecutar y aplica la validación esperada.

```
java -jar target/calculadora-notas-1.0.0.jar 14 16 18
java -jar target/calculadora-notas-1.0.0.jar 12.5 15
java -jar target/calculadora-notas-1.0.0.jar 21
```

Alcance de las evidencias. Las figuras 1 a 4 son capturas del visor de archivos reales de configuración y registros guardados; no son una ventana de terminal interactiva. Los archivos .log íntegros permiten contrastar los resultados y los extractos mostrados.

5. Control de versiones y publicación en GitHub

El proyecto se publicó en github.com/GinoSebastianD/IS2-PC3-GG. Se conserva **main** como rama principal y **codex/practica-03** como rama de implementación. La configuración y el código con sus pruebas se registraron en commits separados.

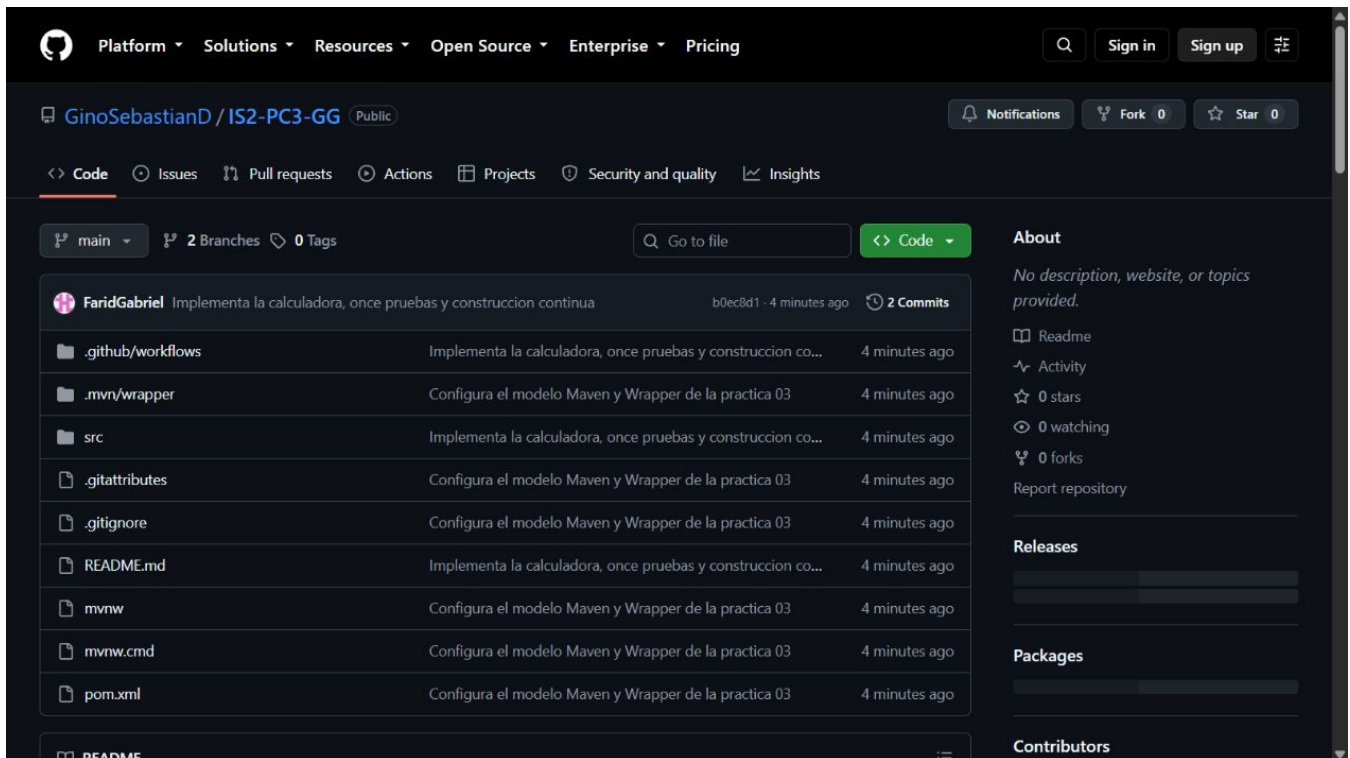


Figura 5. Captura directa del repositorio publicado en GitHub, registrada durante la preparación del informe.

Commits de implementación: f4ef23a configura el modelo Maven y el Wrapper; b0ec8d1 incorpora la calculadora, once pruebas y el flujo de construcción. La documentación y el PDF se agregan en una entrega posterior a esa captura. El historial completo se consulta en GitHub.

```
git switch -c codex/practica-03
git add src .github README.md
git commit -m "Implementa la calculadora, once pruebas y construccion continua"
git switch main
git merge --ff-only codex/practica-03
git push -u origin main codex/practica-03
```

La entrega incluye pom.xml, código Java, pruebas, scripts del Wrapper, tutorial README, este informe y sus capturas. target/ y las cachés se excluyen del control de versiones porque se regeneran con Maven. Los scripts oficiales del Wrapper conservan sus avisos de licencia.

6. Automatización adicional y conclusiones

El flujo **Construccion Maven** se ejecutó correctamente en GitHub Actions sobre Java 17. La ejecución **34721731701**, iniciada manualmente para verificar la entrega, confirma la construcción en un entorno remoto. El archivo `.github/workflows/build.yml` también contempla push y pull_request, y define descarga del código, configuración de Java, clean install, ejecución del JAR y conservación de artefactos.

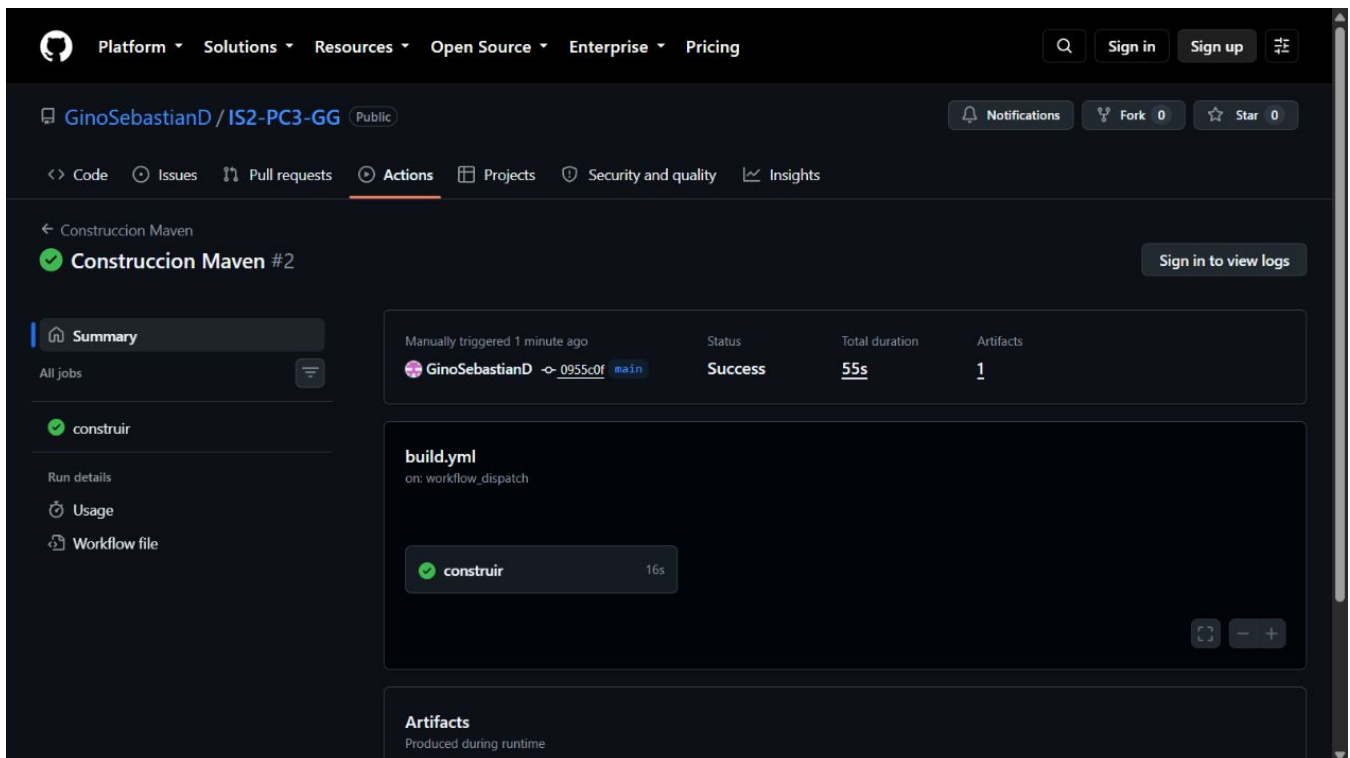


Figura 6. Captura directa del estado de GitHub Actions en la ejecución consultada.

Conclusiones. El modelo de Maven permitió centralizar las opciones del proyecto y construir el software con una sola invocación. La gestión de JUnit y los plugins hizo posible comprobar automáticamente el comportamiento antes del empaquetado. El JAR funcionó con los valores previstos y rechazó una entrada fuera de rango. El Wrapper y el versionado de la configuración facilitan repetir el proceso en otro equipo con un JDK compatible.

Referencias

- [1] Apache Maven. [Maven in 5 Minutes](#). Estructura y configuración inicial.
- [2] Apache Maven. [Maven Wrapper](#). Configuración de la versión de Maven y descarga de la distribución.
- [3] Apache Maven. [Introduction to the Build Lifecycle](#). Fases de construcción e instalación.
- [4] JUnit. [JUnit 5.11.4 User Guide](#). Pruebas y parametrización.

Enunciado del curso: Pr_3_Automatic_Build.pdf, Práctica 03, Construcción automática. Evidencias de ejecución y código: repositorio de la entrega. Consulta y ejecución: 12/09/2026.